# Several GPUs for domain-wall fermions in RutticeQCD

Wei-Lun Chen

October 24, 2026 (internal note)

## Contents

**1 Which data a GPU needs from its neighbour** **2**
1.1 Only two spin components travel . . . 2
1.2 The sender applies the backward link . . . 2
1.3 The fifth dimension is applied before sending . . . 3

**2 Why the result is the same bit for bit** **3**
2.1 Floating-point addition is not associative . . . 3
2.2 The order of the eight legs . . . 3

**3 How a face is numbered** **3**
3.1 The site number of the device fields . . . 3
3.2 The face number . . . 4
3.3 On the sites of one parity . . . 4
3.4 Order in the buffer, and coalescing . . . 4

**4 How the faces travel** **5**
4.1 The sequence . . . 5
4.2 NVSHMEM: an evaluation (not implemented) . . . 5

**5 The gauge field: an extended lattice** **6**

**6 The coarse lattice** **6**
6.1 Aggregates inside a process . . . 6
6.2 Probing across a cut . . . 7
6.3 Ghost sites . . . 7
6.4 The exact adjoint . . . 7
6.5 The levels below . . . 7

**7 Sums over the processes** **8**

**8 What was checked** **8**

# 1 Which data a GPU needs from its neighbour

The hopping term at site $x$ is

$$(H\psi)(x) = -\kappa \sum_{\mu=0}^{3} \Big[ \underbrace{(1 - s\gamma_\mu) U_\mu(x)\, \psi(x+\hat{\mu})}_{\text{forward leg}} + \underbrace{(1 + s\gamma_\mu) U_\mu(x-\hat{\mu})^\dagger\, \psi(x-\hat{\mu})}_{\text{backward leg}} \Big], \tag{1.1}$$

with $s = +1$ for $H$ and $s = -1$ for $H^\dagger$.
A GPU holds the sites $0 \le x_\mu < L_\mu$. For a site with $x_\mu = L_\mu - 1$ the forward leg needs $\psi(x+\hat{\mu})$, which is on the next GPU. For $x_\mu = 0$ the backward leg needs $\psi(x-\hat{\mu})$ and the link $U_\mu(x-\hat{\mu})$, both on the previous GPU.

## 1.1 Only two spin components travel

Take $\mu = 0$. In the Dirac representation used by the kernels,

$$\gamma_0 = \begin{pmatrix} 0 & 0 & 0 & -i \\ 0 & 0 & -i & 0 \\ 0 & i & 0 & 0 \\ i & 0 & 0 & 0 \end{pmatrix}. \tag{1.2}$$

Apply $1 - s\gamma_0$ to $w = (w_0, w_1, w_2, w_3)$:

$$\big[(1 - s\gamma_0)w\big]_0 = w_0 - s(-i)w_3 = w_0 + is\, w_3, \tag{1.3}$$
$$\big[(1 - s\gamma_0)w\big]_1 = w_1 - s(-i)w_2 = w_1 + is\, w_2, \tag{1.4}$$
$$\big[(1 - s\gamma_0)w\big]_2 = w_2 - s(i)w_1 = w_2 - is\, w_1, \tag{1.5}$$
$$\big[(1 - s\gamma_0)w\big]_3 = w_3 - s(i)w_0 = w_3 - is\, w_0. \tag{1.6}$$

Compare row 2 with row 1:

$$-is\,\big[(1 - s\gamma_0)w\big]_1 = -is\, w_1 - is \cdot is\, w_2 \tag{1.7}$$
$$= -is\, w_1 - i^2 s^2\, w_2 \tag{1.8}$$
$$= -is\, w_1 + w_2 \qquad (i^2 = -1,\ s^2 = 1) \tag{1.9}$$
$$= \big[(1 - s\gamma_0)w\big]_2. \tag{1.10}$$

In the same way row $3 = -is\times$ row 0. So rows 2 and 3 follow from rows 0 and 1, and we send only

$$h_a = \big[(1 - s\gamma_\mu)w\big]_a, \qquad a = 0, 1, \tag{1.11}$$

two colour vectors: $2 \times 3$ complex numbers $= 12$ reals instead of 24. The receiver rebuilds rows 2 and 3. For $\gamma_3 = \mathrm{diag}(1, 1, -1, -1)$ it is even simpler:

$$(1 - \gamma_3)w = (0,\, 0,\, 2w_2,\, 2w_3), \qquad (1 + \gamma_3)w = (2w_0,\, 2w_1,\, 0,\, 0). \tag{1.12}$$

## 1.2 The sender applies the backward link

The backward leg needs $U_\mu(x-\hat{\mu})$, a link of the sending GPU. So the sender computes

$$t_a = U_\mu(y)^\dagger\, \big[(1 + s\gamma_\mu)w(y)\big]_a, \qquad y = x - \hat{\mu}, \tag{1.13}$$

and the receiver uses $t_a$ as it is. For the forward leg the sender sends

$$h_a = \big[(1 - s\gamma_\mu)w(y)\big]_a, \qquad y = x + \hat{\mu}, \tag{1.14}$$

and the receiver multiplies by its own $U_\mu(x)$. No GPU needs a link of another.

### 1.3 The fifth dimension is applied before sending

For domain-wall fermions the hop acts on

$$w_s = b\,\psi_s - c\,(Y\psi)_s, \tag{1.15}$$

$$(Y\psi)_s = -P_-\psi_{s+1} - P_+\psi_{s-1} \qquad (+m \text{ instead of } -1 \text{ across the walls}). \tag{1.16}$$

$w_s$ needs $\psi_{s\pm1}$ of the same four-dimensional site, which the sender has. So the sender computes $w$ at its face sites, then projects. All $L_s$ values of $s$ of a site go in one message. Bridge++ does the same (its `5dir` before `hop1`).

## 2 Why the result is the same bit for bit

### 2.1 Floating-point addition is not associative

In double precision:

$$(10^{16}+1) - 10^{16} = 10^{16} - 10^{16} = 0 \qquad (10^{16}+1 \text{ rounds to } 10^{16}), \tag{2.1}$$

$$10^{16} + (1 - 10^{16}) = 10^{16} - (10^{16} - 1) = 1. \tag{2.2}$$

The same three numbers give 0 or 1. Usually only the last bits differ, but they do differ.

### 2.2 The order of the eight legs

One GPU sums the legs $\ell_0, \ldots, \ell_7$ of a site in a fixed order:

$$S_{\text{one}} = (((\ell_0 + \ell_1) + \ell_2) + \cdots) + \ell_7. \tag{2.3}$$

Bridge++ on several GPUs sums the local legs first (bulk kernel) and adds the remote ones later (boundary kernel). For a site whose legs 1 and 5 are remote:

$$S_{\text{Bridge++}} = (\ell_0 + \ell_2 + \ell_3 + \ell_4 + \ell_6 + \ell_7) + (\ell_1 + \ell_5) \;\neq\; S_{\text{one}} \quad \text{in the last bits.} \tag{2.4}$$

We compute such a site entirely in the surface kernel, in the order of $S_{\text{one}}$, reading $\ell_1, \ell_5$ from the received buffer:

$$S_{\text{ours}} = (((\ell_0 + \ell_1^{\text{buf}}) + \ell_2) + \cdots) + \ell_7. \tag{2.5}$$

Each leg is the same arithmetic in both cases (projection on the sender, link product, rebuild on the receiver), so $\ell_1^{\text{buf}} = \ell_1$ exactly and $S_{\text{ours}} = S_{\text{one}}$ exactly. The interior sites (no remote leg) are computed while the faces travel; no site is computed twice.

## 3 How a face is numbered

### 3.1 The site number of the device fields

A site is numbered lexically, $x$ fastest:

$$n(x,y,z,t) = x + L_x(y + L_y(z + L_z\,t)). \tag{3.1}$$

For the face $t = L_t - 1$:

$$n = \underbrace{L_xL_yL_z(L_t-1)}_{\text{offset}} + (x + L_x(y + L_yz)), \tag{3.2}$$

one contiguous range. For the face $x = L_x - 1$:

$$n = (L_x - 1) + L_x(y + L_y(z + L_z\,t)), \tag{3.3}$$

so consecutive $y$ give sites $L_x$ apart. Three of the four faces are not contiguous, and a kernel must gather them.

### 3.2 The face number

Let $d_0 < d_1 < d_2$ be the directions other than $\mu$. We number a site of the face by

$$f = c_{d_0} + L_{d_0}(c_{d_1} + L_{d_1}\, c_{d_2}). \tag{3.4}$$

Two neighbours across the face differ only in $c_\mu$:

$$x = (\dots, c_\mu = L_\mu - 1, \dots) \text{ on GPU } A, \qquad x + \hat{\mu} = (\dots, c_\mu = 0, \dots) \text{ on GPU } B, \tag{3.5}$$

with the same $c_{d_0}, c_{d_1}, c_{d_2}$. Hence $A$ and $B$ compute the same $f$ from Eq. (3.4). Nothing else has to be agreed on: no table is kept or sent.

### 3.3 On the sites of one parity

The parity of a site is

$$p = (c_\mu + c_{d_0} + c_{d_1} + c_{d_2}) \bmod 2. \tag{3.6}$$

For fixed $c_\mu, c_{d_1}, c_{d_2}$, solve for $c_{d_0}$:

$$c_{d_0} \equiv p - (c_\mu + c_{d_1} + c_{d_2}) \pmod 2, \tag{3.7}$$

$$c_{d_0} = 2k + r, \qquad r = (p + c_\mu + c_{d_1} + c_{d_2}) \bmod 2, \qquad k = 0, \dots, \tfrac{L_{d_0}}{2} - 1. \tag{3.8}$$

So exactly one site of each pair $(2k, 2k+1)$ has the parity, and $k = \lfloor c_{d_0}/2 \rfloor$ numbers it:

$$f = \lfloor c_{d_0}/2 \rfloor + \tfrac{L_{d_0}}{2}(c_{d_1} + L_{d_1}\, c_{d_2}). \tag{3.9}$$

This needs $L_{d_0}$ even. We require all local extents even. Then the local parity is also the global one:

$$\sum_\mu (\text{origin}_\mu + c_\mu) \equiv \sum_\mu c_\mu \pmod 2, \qquad \text{origin}_\mu = k_\mu L_\mu \text{ even}. \tag{3.10}$$

### 3.4 Order in the buffer, and coalescing

The buffer of a face holds, for vector $v$, fifth coordinate $s$, real $r$ $(0 \le r < 12)$ and face site $f$,

$$\text{index}(v, s, r, f) = \big((v\, L_s + s) \cdot 12 + r\big) \cdot N_f + f. \tag{3.11}$$

In the packing kernel thread $j$ of a warp has $f = f_0 + j$. For fixed $(v, s, r)$:

$$\text{index}(f_0 + j) = \text{index}(f_0) + j, \qquad j = 0, \dots, 31 : \tag{3.12}$$

32 consecutive reals, one memory transaction for the warp.

The reads from the field follow the site number. A field has component $c$ of site $n$ at

$$a(c, n) = \big(\lfloor n/32 \rfloor\, N_c + c\big) \cdot 32 + (n \bmod 32). \tag{3.13}$$

- Faces in $y, z, t$: $d_0 = x$, so $f_0 + j$ has $x_0 + j$ and by Eq. (3.1) $n = n_0 + j$; within a block of 32, $a(c, n_0 + j) = a(c, n_0) + j$: coalesced.
- Face in $x$: $d_0 = y$, so $f_0 + j$ has $y_0 + j$ and $n = n_0 + jL_x$; the addresses are $L_x$ apart: not coalesced.

Only the face in $x$ is read with a stride. It has $V/L_x$ sites like the others, and packing is small next to the interior, so we left it.

The fused hop-and-matrix kernel uses an XOR on the column of a tile in shared memory, `column ^ (8*(row & 1))`, against bank conflicts. The face buffers are in global memory and read along $f$, so no XOR is needed there.

# 4 How the faces travel

## 4.1 The sequence

For one application of the hops:

1. pack every cut face (default stream)
2. send and receive the faces
3. interior kernel, at the same time as 2
4. wait for 2
5. surface kernel

**NCCL**: step 2 runs on a stream of its own; an event after step 1 starts it, an event after it makes step 5 wait. The host waits for nothing.

**Through the host**: step 2 is copy to the host, MPI exchange, copy back; the interior kernel runs while MPI exchanges.

**Choice**: NCCL if every process can make its communicator:

$$\text{failed} = \sum_{\text{processes}} \big[\text{NCCL failed here}\big], \qquad \text{use NCCL} \iff \text{failed} = 0, \tag{4.1}$$

so all processes choose the same.

**Two messages between one pair.** If a direction is cut in two, GPU $A$ sends GPU $B$ its face forward and its face backward. NCCL matches the messages of a pair in the order they are posted. Posting the sends in the order of the faces $f = 0, \ldots, 7$ and the receives in the order of the faces the data left by keeps the two orders the same on both sides.

## 4.2 NVSHMEM: an evaluation (not implemented)

**A model of one application.** Let an operator take $t_c$ to compute on a GPU and an exchange $t_x = t_0 + b/B$: a fixed latency $t_0$ and $b$ bytes at bandwidth $B$. With the interior overlapping the exchange,

$$t_{\text{apply}} \approx t_{\text{surface}} + \max\big(t_{\text{interior}},\ t_x\big). \tag{4.2}$$

Two limits follow:

$$t_{\text{interior}} \gg t_x: \quad t_{\text{apply}} \approx t_c \quad \text{(the exchange is hidden; its kind does not matter)}, \tag{4.3}$$

$$t_{\text{interior}} \ll t_x: \quad t_{\text{apply}} \approx t_x \approx t_0 \quad \text{(the latency is all)}. \tag{4.4}$$

**Where each case is.** The fine operator on a large local lattice is the first case. A local lattice $16^4$ with $L_s = 16$ has

$$\text{interior work} \propto 16^4 \cdot 16 \approx 10^6 \text{ site-slices, about a millisecond}, \tag{4.5}$$

$$\text{a face} = 16^3 \cdot 16 \cdot 12 \text{ reals} \cdot 8 \text{ B} \approx 6 \text{ MB, about 0.1 ms on NVLink}, \tag{4.6}$$

so the exchange hides. The coarse lattice is the second case. With blocks of $4^4$ the coarse local lattice is $4^4 = 256$ sites; a face is 64 sites of dim $\approx 40$ complex numbers,

$$64 \cdot 40 \cdot 16 \text{ B} \approx 40 \text{ kB}, \qquad t_c \sim 10\ \mu\text{s}, \tag{4.7}$$

and the coarse solve applies the operator hundreds of times per fine iteration.

**Latencies (typical, not measured here).** An NCCL group of sends and receives costs a launch and its protocol, $t_0 \sim 10$–$20\ \mu$s; an NVSHMEM put with a signal within a node a few $\mu$s. On the coarse lattice then

$$\frac{t_{\text{apply}}^{\text{NCCL}}}{t_{\text{apply}}^{\text{NVSHMEM}}} \approx \frac{t_c + 15\ \mu\text{s}}{t_c + 4\ \mu\text{s}} \approx \frac{25}{14} \approx 1.8, \tag{4.8}$$

a 1.5–3× faster coarse solve depending on $t_c$; on the fine operator a few per cent at most.

**Why NVSHMEM can be faster there.** (i) No group launch: the put is issued on the stream, or by the packing kernel itself. (ii) Within a node the packing kernel can write straight into the neighbour's buffer through a peer pointer (`nvshmem_ptr`): the face is never copied a second time, and our layout ($f$ fastest) already makes those stores coalesced. (iii) The wait is a signal on the stream (or in the kernel), not a host round trip. (iv) NCCL's kernels occupy multiprocessors while they move data; NVSHMEM's puts use the copy engines or the network.

**What it would take.** Receive buffers and signals on the symmetric heap (one collective allocation, sized for the largest number of vectors, doubled so a GPU one step ahead never overwrites what its neighbour reads); a feature `nvshmem`; bootstrap by the same broadcast of a unique identifier as NCCL; first the stream-ordered form `putmem_signal_on_stream`/`signal_wait_until_on_stream`, then the direct stores within a node. NCCL and NVSHMEM can live in one program (NCCL for the fine operator and the sums, NVSHMEM for the coarse ghosts); the symmetric heap should then be kept small (`NVSHMEM_SYMMETRIC_SIZE`). Recent NCCL versions add symmetric memory and device-initiated operations of their own, which might give the same without NVSHMEM; that has to be checked against their documentation first.

**When to do it.** Only on a machine with several GPUs, where it can be measured: our machine has one, so neither path can be timed here.

# 5 The gauge field: an extended lattice

A staple at $x$ in direction $\mu$, plane $\nu$, reads

$$U_\nu(x)\,U_\mu(x+\hat\nu)\,U_\nu(x+\hat\mu)^\dagger + U_\nu(x-\hat\nu)^\dagger\,U_\mu(x-\hat\nu)\,U_\nu(x-\hat\nu+\hat\mu). \tag{5.1}$$

The last link sits at $x-\hat\nu+\hat\mu$, a diagonal neighbour. A rectangle reaches $x+2\hat\mu+\hat\nu$.

We copy the links into an extended lattice with a halo of depth $d$ ($d=1$ staples, $d=2$ rectangles):

$$-d \le c_\mu < L_\mu + d \quad \text{in every cut direction.} \tag{5.2}$$

The halo is filled one direction after the other. With two cut directions, $x$ then $y$:

$$\begin{aligned}
&\text{after } x: \quad \text{filled} = \{-d \le c_x < L_x + d,\ 0 \le c_y < L_y\},\\
&\text{send in } y \text{ the layers } 0 \le c_y < d \text{ and } L_y - d \le c_y < L_y, \text{ for all } -d \le c_x < L_x + d,\\
&\text{after } y: \quad \text{filled} = \{-d \le c_x < L_x + d,\ -d \le c_y < L_y + d\}.
\end{aligned}$$

The corners ($c_x < 0,\ c_y < 0$) came with the $y$ layers, which already carried the $x$ halo; no diagonal message is needed. This is the CPU's `gauge::paths::Extended` on the device.

The kernels compute only the local sites and read the extended field; every path from a local site stays inside it. The arithmetic of a staple is unchanged, so the result is that of one GPU bit for bit. The fermion force uses the face exchange of the hops with $b=1$, $c=0$, i.e. $w=\psi$.

# 6 The coarse lattice

## 6.1 Aggregates inside a process

With a block size $B_\mu$ dividing the local extent $L_\mu$:

$$L_\mu = k_\mu B_\mu \quad \Rightarrow \quad \text{blocks } \big[jB_\mu, (j+1)B_\mu\big),\ j = 0, \ldots, k_\mu - 1, \tag{6.1}$$

none crosses $c_\mu = L_\mu$. Restriction and prolongation stay local.

### 6.2 Probing across a cut

The link from coarse site $X$ to its neighbour in direction $\mu$ is

$$B_\mu(X)_{ij} = \left[P^\dagger\, H_\mu\, P\, e_j\right]_i(X), \tag{6.2}$$

with $H_\mu$ the forward leg alone and $e_j$ the unit vector of component $j$ on every coarse site. $H_\mu$ at a fine site of $X$ on the face reads $Pe_j$ on the next GPU: a face exchange of the field $Pe_j$, done once per batch of $j$ and used for all eight legs.

### 6.3 Ghost sites

The coarse kernel reads a neighbour through a table $N[d][X]$. On several processes it points past the local sites:

$$N[d][X] = \begin{cases} \text{local neighbour}, & \text{inside the process}, \\ n_{\text{sites}} + \text{first}[d] + f(X), & \text{across a cut}, \end{cases} \qquad \text{first}[d] = \sum_{d'<d} \text{count}[d'], \tag{6.3}$$

with $f$ the face number of Eq. (3.4) on the coarse lattice. Before each application the faces of the input are gathered, exchanged and placed at those indices of a separate ghost field. The vectors keep only local sites, so an inner product is a local sum followed by a sum over processes.

### 6.4 The exact adjoint

For $(Av)(X) = A(X)v(X) + \sum_d B_d(X)\, v(N[d][X])$:

$$\langle u, Av\rangle = \sum_X u(X)^\dagger \Big[A(X)v(X) + \sum_d B_d(X) v(N[d][X])\Big] \tag{6.4}$$

$$= \sum_X \left[A(X)^\dagger u(X)\right]^\dagger v(X) + \sum_d \sum_X \left[B_d(X)^\dagger u(X)\right]^\dagger v(N[d][X]). \tag{6.5}$$

In the second term put $Y = N[d][X]$. Then $X$ is the site whose hop $d$ comes from $Y$, i.e. $X = N[\bar d][Y]$ with $\bar d$ the opposite direction:

$$\langle u, Av\rangle = \sum_Y \Big[A(Y)^\dagger u(Y) + \sum_d B_d(N[\bar d][Y])^\dagger u(N[\bar d][Y])\Big]^\dagger v(Y), \tag{6.6}$$

so

$$(A^\dagger u)(Y) = A(Y)^\dagger u(Y) + \sum_d B_d(N[\bar d][Y])^\dagger\, u(N[\bar d][Y]). \tag{6.7}$$

When $N[\bar d][Y]$ is a ghost its link $B_d$ lives on the neighbouring process. We exchange the links of the face sites once (at setup) and form the conjugate transposes; a conjugate transpose is a copy with signs, so the numbers are those of one GPU.

### 6.5 The levels below

Each level is a lattice cut on the same process grid; a super-block of $b$ sites per direction must divide the local extent of the level above, so it stays inside a process.

Its operator is probed with colours. With $k_\mu$ the smallest divisor $\geq 3$ of the global extent,

$$\text{colour}(c) = \sum_\mu (c_\mu \bmod k_\mu) \prod_{\nu<\mu} k_\nu, \tag{6.8}$$

of the *global* coordinates $c$. A ghost stands for a site of the neighbour whose global coordinates we know, so it gets that site's colour, and probing finds the links across the cut.

The last level is solved with its dense inverse. Every process gathers the whole level: each puts its sites at their global places in an array of zeros, and the arrays are summed over processes,

$$\text{whole}[g] = \sum_p \text{part}_p[g], \qquad \text{part}_p[g] = \begin{cases} \text{value}, & g \text{ on } p, \\ 0, & \text{otherwise}, \end{cases} \tag{6.9}$$

and value $+ 0 + \cdots + 0 =$ value exactly. Every GPU then inverts the same matrix. (The CPU does the same for its levels.)

# 7 Sums over the processes

An inner product on $P$ processes is

$$\langle a, b \rangle = \sum_{p=1}^{P} \Big( \sum_{x \in p} a(x)^* b(x) \Big), \tag{7.1}$$

another order of the same terms than one sum over all $x$. By Sec. 2.1 the last bits differ, so a solver is not bit for bit. Example: CGNE to $|r|^2/|b|^2 < 10^{-22}$ stopped after 1247 iterations on two and four processes and 1246 on one; the solutions differ by $10^{-11}$ relative. The operators, which have no such sum, are bit for bit.

# 8 What was checked

Tests run several processes as threads of one program, all on the one A30, against one process:

| what | result |
|---|---|
| $D$, $D^\dagger$, $\Gamma_5 D$, $\tilde{A}$ (both forms, $\alpha = 1.3$), up to 16 ranks | bit for bit |
| stout (2 steps), chain rule, rectangle force, fermion force | bit for bit |
| P, coarse operator, $D_c$, $\Gamma_5 D_c \Gamma_5$, $D_c^\dagger$, Schur | bit for bit |
| levels: quark V-cycle, $\lvert H\rvert^{-1}$ of the Pauli-Villars factor | bit for bit |
| CGNE | iterations $\pm 1$, $10^{-11}$ |

Every bit-for-bit test was also broken on purpose (wrong boundary factor, halo too shallow, ghosts on the wrong face, wrong ghost colours, ghost links of the wrong direction) and then failed.

With two MPI processes on $4^3 \times 8$, GPU and CPU alike: the propagator (CG, mixed CG, the multigrid with and without a second level, MINRES or GCR for the Pauli-Villars factor) and HMC (CG, mixed CG, the multigrid with a second level) take the same iterations as one process, $\Delta H$ to $10^{-10}$.

Not checked: NCCL itself and several real GPUs (one GPU here; NCCL refuses two processes on one GPU, so the host path ran), and the speed of the exchanges (on one shared GPU each exchange pays a context switch: HMC with the multigrid 48 s on two processes against 1.6 s on one, which says nothing about several GPUs).